



Technical Report
manageProjectMember — LWC Creation Report

SUBMITTED BY
Maamoun Youssef
Jira Clone
2026-05-30

Component Overview

The manageProjectMember component is a Salesforce Lightning Web Component that provides a full CRUD interface for managing team members on projects. The screenshots below show the deployed component running in a Salesforce Developer Edition org.

1. Project Selection & Member List

When the component loads, a project selector dropdown at the top lets the user choose from all available projects. Once a project is selected, its members are displayed in a list below, each with an edit (✎) and delete (🗑) action button.

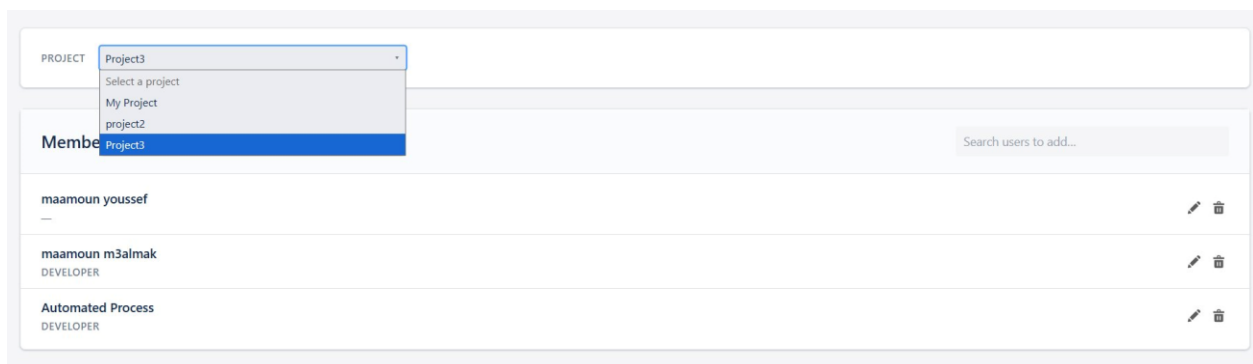


Figure 1 — Project combobox open showing available projects; member list rendered for Project3

2. User Search Autocomplete (Add Member)

Typing at least 2 characters in the "Search users to add..." field triggers a debounced SOSL search (180 ms). Matching users appear as a dropdown list, each showing an avatar with initials, the full name, and their Salesforce User ID. An "+ Add" button next to each result allows immediate addition.

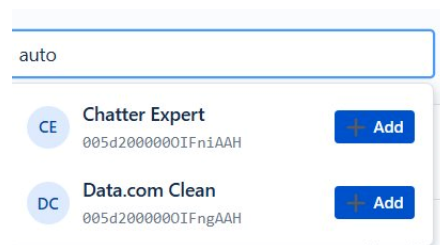


Figure 2 — Search autocomplete showing results for "auto": Chatter Expert and Data.com Clean

3. Member Added — Toast Confirmation

After clicking "+ Add", the new member is immediately appended to the list and a green success toast "[Name] added to project" appears at the bottom-center of the screen. The member count badge on the "Members" heading increments accordingly.

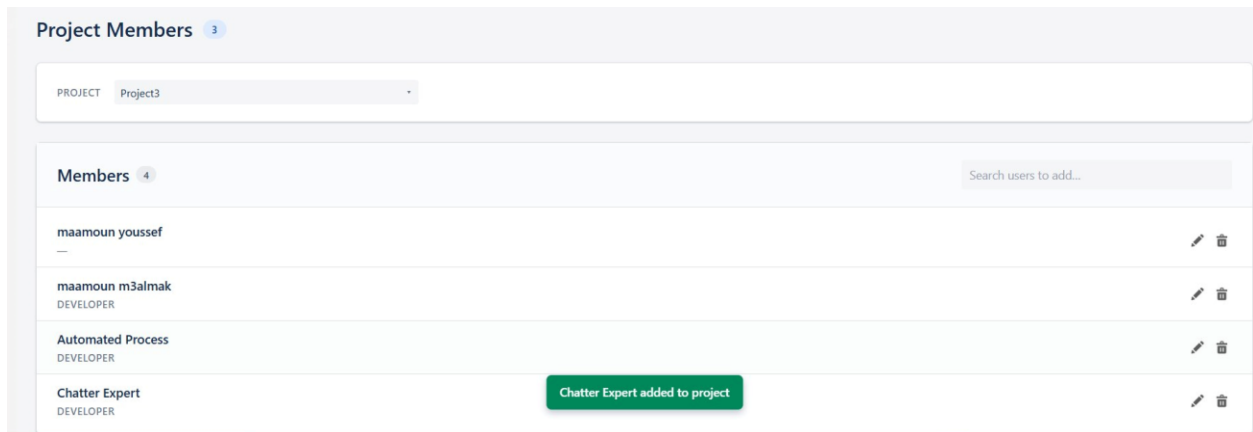


Figure 3 — Chatter Expert successfully added; toast notification visible; member count updated to 4

4. Edit Member Modal

Clicking the edit (✎) icon on any member row opens the "Edit member" modal. The modal displays the member's Account ID (read-only), Display name (read-only, sourced from User.Name via lookup), and a Role combobox pre-populated with the current value. The user can change the role and click "Save changes", or dismiss with "Cancel".

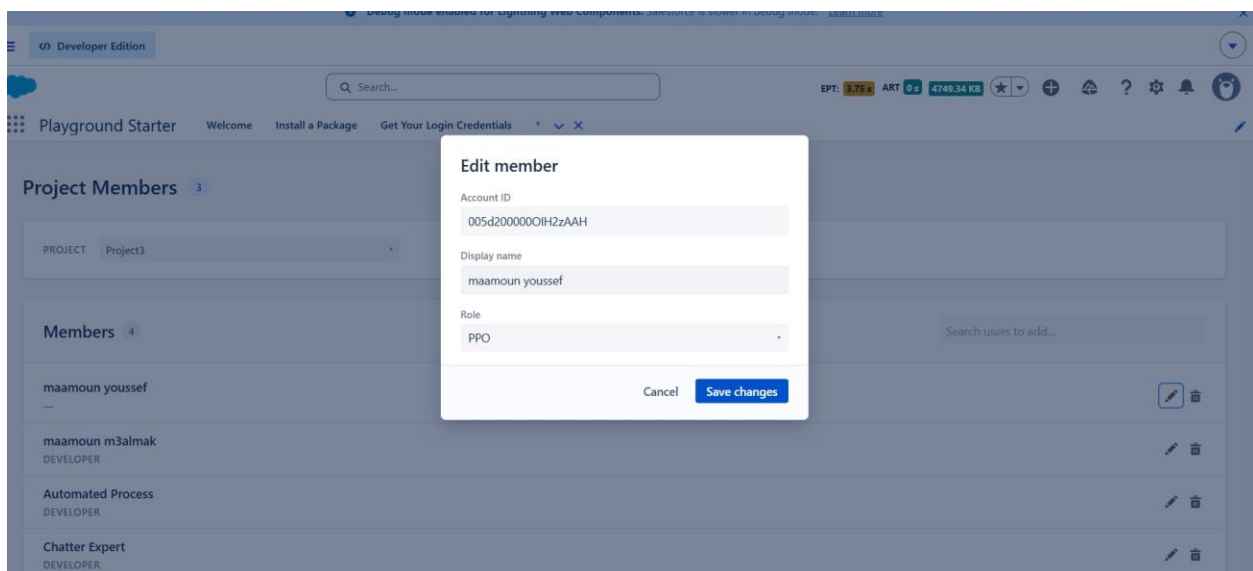


Figure 4 — Edit modal open for "maamoun youssef"; role set to PPO; background list shows all 4 members

1. FAQ — Questions Asked and Answers Received

Listed in the order the AI asked them. Each entry shows the skill step that drove the question, the question itself, and the user's answer.

Q1 — Component Location

Skill step: create-new-parent-lwc-component → Step 1 (scaffold)

Question: "Where should the new parent component manageProjectMember live?"

Answer: Default path — force-app/main/default/lwc/manageProjectMember/

Q2 — Iteration 1 Path Selection

Skill step: create-new-parent-lwc-component → Step 2 (loop tracker)

Question: "For the next iteration on manageProjectMember, which path?"

Options: A (new functionality) / B (handle child event) / C (stop)

Answer: A — Add new functionality. Delegates to add-interactive-with-data-persistence-functionality-in-pather sub-skill.

Q3 — Sub-components + Functionality Label (Step 2.1, first pass)

Skill step: add-interactive-with-data-persistence-functionality-in-pather → Step 2.1

Question: "Which sub-components does this new functionality apply to? (Also, what would you like to call this functionality — e.g. 'addMember', 'removeMember', 'changeRole'?)"

Answer: "I'll describe them" — followed by the full Project Members Manager — State & Behavior Spec: default state, project switch, search with 180ms debounce + 2-char minimum, "+ Add" affordance, edit modal, delete confirmation modal, toast behavior.

Q4 — Sub-components (Step 2.1, re-ask)

Skill step: Step 2.1 (re-ask, discipline: do not infer)

Question: "Which sub-components (custom child LWCs) does this manageMembers functionality apply to?"

Answer: None — base components only.

Q5 — User Stories (Step 2.2)

Skill step: Step 2.2

Question: "Please write the user stories for manageMembers — one per operation."

Answer — Five stories:

1. "As a user, I want to select a project so I can see its team members."
2. "As a user, I want to search for a user so I can add them to the project."
3. "As a user, I want to add a member so I can give them access to the project."
4. "As a user, I want to edit a member so I can update their name or role."
5. "As a user, I want to remove a member so I can revoke their access to the project."

Q6 — Validation Rules (Step 2.4)

Skill step: Step 2.4

Question: "Which validations must run before each Apex call?"

Answer: Use the 4 recommended rules:

- Add — reject if user already a member.
- Edit — role is in the allowed list.
- Delete — user confirmed in modal.
- Search — query length ≥ 2 .

Q7 — Data State (Step 2.5)

Skill step: Step 2.5. AI flagged two discrepancies between the spec and the existing schema (role picklist values, no display-name field).

Answer: Use existing schema as-is.

- Object: ProjectMember__c { Project__c, User__c, Role__c, RecordStatus__c }
- Display name = User.Name via the lookup (read-only in the edit modal)
- Role picklist: PPO, Consultant, Test Factory, Scrum Master, Developer
- Spec adjustment: Step 2.4 (b) reduces to "role in allowed list" — the "name non-empty" check is dropped because name is read-only.

Q8 — Base Components (Step 2.6)

Skill step: Step 2.6

Question: "Which existing base components should manageProjectMember reuse instead of writing UI from scratch?"

Answer: c-ao-input + c-ao-btn + c-ao-combobox (all three).

Q9 — Apex Method Resolution Branch (Step 2.7)

Skill step: Step 2.7. AI inventoried existing service methods first and identified 3 missing.

Question: "We need 5 @AuraEnabled methods on the parent. Which branch?"

Answer: C — Create new ManageProjectMemberController.

Q10 — Confirm Controller Method Specs + Location

Skill step: Step 2.7 Creation Sub-Loop (Branch C)

Answer: Confirm + default path.

- Controller: force-app/main/default/classes/controller/manageProjectMember/ManageProjectMemberController.cls
- Services live in existing force-app/main/default/classes/domain/ProjectService.cls

Feedback Intervention — SOSL, not SOQL, for User Search

Trigger: AI's first attempt to extend ProjectService.cls used SOQL Name LIKE '%query%'.

User's correction: "no you use soql you should use sosl for search by term and change the method name to be search by term. you can monitor : loadTicketBySearchTerm in manageBacklogController in line 59"

AI response: Read ManageBacklogController.loadTicketBySearchTerm (lines 59–114), adopted the same SOSL pattern (Search.query with special-character escaping), and renamed the method to searchUsersByTerm. The "already a member" filter happens in-memory after the SOSL call.

Q11 — Iteration 2 Path Selection

Skill step: create-new-parent-lwc-component → Step 2 (loop tracker)

Answer: C — Stop and finalise.

Q12 — Apex Method Monitor

Skill step: apex-method-monitor (post-controller hook from CLAUDE.md)

Question: "Create a governor-limit monitoring report for the 6 new @AuraEnabled methods on ManageProjectMemberController?"

Answer: No — skip monitoring.

2. Files Read by the AI — In Order

Chronological list of every file the AI inspected, with a one-line note on why.

#	Path	Purpose
1	force-app/skills/create-new-parent-lwc-component-skill.md	Read the entry-point skill the IDE had open.
2	force-app/main/default/lwc/manageProjectMember/**/* (glob)	Confirm no existing component at this name.
3	force-app/main/default/lwc/manageBacklog/* (glob)	Inspect sibling parent's file layout.
4	force-app/main/default/lwc/manageWorkflow/* (glob)	Inspect another sibling parent's file layout.
5	manageWorkflow/manageWorkflow.js-meta.xml	Match the project's .js-meta.xml conventions for the new scaffold.

#	Path	Purpose
6	add-interactive-with-data-persistence-functionality-in-pather-skill.md	Path A delegation target — read full sub-skill before running the interview.
7	objects/ProjectMember__c/fields/*.field-meta.xml (glob)	Enumerate the data-state object's fields.
8	objects/Project__c/fields/*.field-meta.xml (glob)	Confirm Project__c has only BacklogMaxScore__c custom.
9	fields/User__c.field-meta.xml	Confirm User__c is a Lookup to User.
10	fields/Role__c.field-meta.xml	Discover the actual Role picklist values — drove the Step 2.5 discrepancy flag.
11	ls force-app/main/default/lwc/ filter ^ao* (bash)	Enumerate available base components for Step 2.6.
12	ls force-app/main/default/classes/controller/ (bash)	See controller folder convention.
13	ls ../controller/chooseproject/ (bash)	See an existing thin controller's file layout.
14	grep -l "ProjectMember" ../classes/ (bash)	Locate any class already touching ProjectMember__c.
15	grep "ProjectMember" ProjectService.cls (bash)	Identify the 4 existing ProjectMember service methods.
16	classes/domain/ProjectService.cls	Read the existing service to plan additions and reuse.
17	controller/chooseproject/ProjectSplashController.cls	Confirm thin-controller pattern: try/catch → service → APIResponse.
18	OBJECT_VALIDATION_LWC_APEX.md	Apply object-required-fields rule for ProjectMember__c.
19	ls force-app/main/default/classes/ (bash)	Locate the shared/ folder.
20	find APIResponse* ServiceException* (bash)	Resolve the envelope + exception types.

#	Path	Purpose
21	classes/shared/APIResponse.cls	Confirm APIResponse(success, message, data) constructor signature.
22	ls force-app/main/default/lwc/manageBacklog (bash)	Locate manageBacklog reference files.
23	manageBacklog/manageBacklog.js (lines 1–100)	Study a real parent's wire/import/state conventions.
24	grep "localStorage projectId" manageBacklog.js (bash)	Decide whether this parent needs the localStorage projectId entry.
25	ls force-app/skills/ (bash)	Discover the CSS design skill.
26	lwc-css-design-skill.md (lines 1–60)	Read the design-system overview (palette, fonts).
27	lwc-css-design-skill.md (lines 60–260)	Read color / typography / spacing / state recipes.
28	lwc/aoBtn/aoBtn.js	Learn the c-ao-btn API (variant, size, icon-name, label, title).
29	lwc/aolInput/aolInput.js	Learn the c-ao-input API (value, variant, change event shape).
30	lwc/aoCombobox/aoCombobox.js	Learn the c-ao-combobox API (options, value, change event shape).
31	grep "modal scrim overlay" manageBacklog.css (bash)	Locate reusable modal CSS classes.
32	manageBacklog/manageBacklog.css (lines 100–200)	Copy modal-backdrop + modal recipes verbatim.
33	ManageBacklogController.cls (lines 50–130)	After user feedback: study loadTicketBySearchTerm (line 59) for SOSL pattern.
34	ProjectSplashController.cls-meta.xml	Match the apiVersion / status in the new controller's meta.

#	Path	Purpose
35	ls docs/ (bash)	Confirm docs/ folder before writing this report.

3. Files Written / Modified

All artifacts produced by the AI session as a result of the above decisions.

Action	Path
Created	force-app/main/default/lwc/manageProjectMember/manageProjectMember.js-meta.xml
Created	force-app/main/default/lwc/manageProjectMember/manageProjectMember.js
Created	force-app/main/default/lwc/manageProjectMember/manageProjectMember.html
Created	force-app/main/default/lwc/manageProjectMember/manageProjectMember.css
Created	force-app/main/default/lwc/manageProjectMember/manageProjectMemberValidator.js
Created	force-app/main/default/classes/controller/manageProjectMember/ManageProjectMemberController.cls
Created	force-app/main/default/classes/controller/manageProjectMember/ManageProjectMemberController.cls-meta.xml
Modified	force-app/main/default/classes/domain/ProjectService.cls — added searchUsersByTerm (SOSL), updateMemberRole, softDeleteMember; added Role__c to loadProjectMembers SELECT list.